# Java OOP Laboratory Exercise

Campus Café Ordering System

**Name:** Callora
**Section:** D
**Date:** November 10, 2026

## 1. Concept Explanation

| Concept | Where and why it is used |
|---|---|
| **if Statement** | Pastry.computePrice() adds the warming fee only if warmed; Meal.computePrice() adds the upsize fee only if upsized. A single if is enough because there is just one extra condition. |
| **if-else Statement** | Used in MenuItem.setBasePrice() (reject negatives), Beverage.computePrice() (size chain), Order.addItem() (first empty slot or full) and Order.computeTotal() (student or not), because each choice has two or more outcomes. |
| **Classes and Objects** | Seven classes, each in its own file; CafeApp.main() creates the customers, items and orders so every class is actually used. |
| **Attributes and Behaviors** | Every class keeps its data in instance variables (e.g., size, warmed, loyaltyPoints) and works on it through instance methods. |
| **Object Interaction** | Order stores a Customer and up to three MenuItem objects; printReceipt() calls each item's methods and updates the Customer's loyalty points. |
| **Naming Conventions** | PascalCase for classes, camelCase for methods and variables, UPPER_SNAKE_CASE for constants such as WARMING_FEE and PESOS_PER_POINT. |
| **Default Constructor** | MenuItem(), Beverage(), Pastry(), Meal() and Customer() set safe starting values so an object can be built first and filled in later with setters. |
| **Parameterized Constructor** | All MenuItem subclasses, Customer and Order accept their data at creation so objects start complete and valid. |
| **Constructor Overloading** | Customer has three constructors (none, name, name + student); the shorter ones call the full one with this(...) to avoid repeated code. |
| **public / private / protected** | Fields are private, except name and basePrice in MenuItem, which are protected so subclasses can use them in computePrice(). Methods that other classes call are public. |
| **Getters, Setters, Data Hiding** | No public fields; setters validate with if/if-else (e.g., no negative prices, sizes only S/M/L, points never negative). |
| **Method Overloading** | Customer.addPoints(int) / addPoints(double) and Order.computeTotal() / computeTotal(double): same action, different inputs. |
| **Method Overriding** | Beverage, Pastry and Meal override computePrice() and getDescription() with @Override, so each type prices and describes itself. |
| **Abstraction** | MenuItem is abstract with the abstract method computePrice(); it cannot be instantiated and forces every subclass to define its own pricing. |
| **Polymorphism** | Order.computeSubtotal() calls computePrice() on MenuItem references; Java runs the Beverage, Pastry or Meal version at run time. |

## 2. Program Output

```
$ javac *.java && java CafeApp
Order is full. Cannot add Cookie [P-02].
Order #1001 - Ana Reyes (Student)
Iced Latte [B-01] Size: L ..................... 120.00
Ensaymada [P-01] (Warmed) ..................... 55.00
Chicken Adobo [M-01] Upsized, 1 side ........ 142.00
Subtotal: 317.00
Student discount (10%): -31.70
TOTAL: 285.30
Loyalty points: 5

Error: base price cannot be negative.
Order #1002 - Ben Cruz
Hot Chocolate [B-02] Size: M .................. 95.00
Beef Tapa [M-02] Regular, 0 sides ........... 130.00
Subtotal: 225.00
Voucher: -20.00
TOTAL: 205.00
Loyalty points: 4
```

## 3. Challenges Encountered and Solutions

• **No arrays or ArrayList.** Order needed to hold up to three items without a collection. I used three fields (item1, item2, item3) and an if / else-if chain in addItem() that fills the first empty slot and refuses the fourth item.

• **Loyalty points from money spent.** One point per 50 pesos meant dropping the decimal part. Casting (int) (amount / 50) gave 5 points for 285.30 and 4 for 205.00, matching the sample.

• **Points awarded twice.** printReceipt() could be called more than once and would add points each time, so I added a pointsAwarded flag to award them only once per order.

• **Aligned receipt lines.** Prices needed to line up after the description, so a private helper pads each line with dots to a fixed width constant.

• **Validation inside constructors.** To avoid calling overridable setters from constructors, the constructors repeat a short if-else check on the same rules the setters use.